Solving Tensor Linear Systems under the t-product by a Kaczmarz-type method

Jeremie Mabiala

Kera Health, Inc.

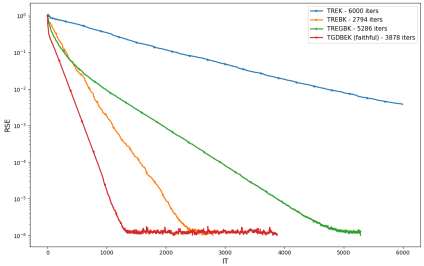
AIMIC, Doctoral Training School - 2026

Why Tensors?

- Images, videos, and multi-dimensional data are naturally represented as **third-order tensors** $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$.
- **Image reconstruction**: given a blurred/noisy observation $\mathcal{B}$, recover the clean image $\mathcal{X}$ from

$$\mathcal{A} * \mathcal{X} = \mathcal{B}$$

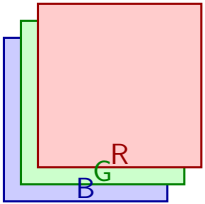
  where $*$ denotes the *t-product*.
- Systems are often **large-scale** and **inconsistent**.



Convergence on `Cities` matrix

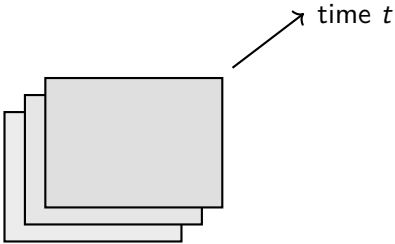
Images and Videos as Tensors

Image Tensor



$\mathcal{I} \in \mathbb{R}^{H \times W \times C}$ with $C = 3$ (RGB)

Video Tensor



$\mathcal{V} \in \mathbb{R}^{H \times W \times C \times T}$

Under the t-product framework, we typically work with third-order tensors; videos can be handled frame-wise or via suitable reshaping.

Matrix Kaczmarz and Randomized Kaczmarz

Consider a linear system $Ax = b$ with rows $a_i^\top$ of A.

Kaczmarz update (row projection):

$$x^{k+1} = x^k + \frac{b_i - a_i^\top x^k}{\|a_i\|_2^2} a_i$$

- At each step, project x^k onto the hyperplane $\{x : a_i^\top x = b_i\}$.
- Cyclic version scans rows in order $i = 1, 2, \ldots, m$.

Randomized Kaczmarz (RK):

- Pick row i_k randomly, often with $\Pr(i_k = i) = \|a_i\|_2^2 / \|A\|_F^2$.
- Same projection formula with $i = i_k$.
- For consistent systems, RK converges *linearly in expectation*.

Bridge to this talk: TGDBEK is the tensor/t-product extension using greedy row/column-slice selection.

Problem Setup

Tensor Linear System

Given $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ and $\mathcal{B} \in \mathbb{R}^{m \times k \times p}$, find $\mathcal{X} \in \mathbb{R}^{n \times k \times p}$ satisfying:

$$\mathcal{A} * \mathcal{X} = \mathcal{B}$$

Goal: compute the *minimum-norm least-squares* solution

$$\mathcal{X}^\star = \underset{\mathcal{X}}{\operatorname{argmin}} \|\mathcal{X}\|_F \quad \text{s.t.} \quad \mathcal{X} \in \underset{\mathcal{Z}}{\operatorname{argmin}} \|\mathcal{A} * \mathcal{Z} - \mathcal{B}\|_F$$

Setting:

- System can be **inconsistent** ($\mathcal{B} \notin \operatorname{range}(\mathcal{A})$)
- Dimensions m, n large $\Rightarrow$ direct solvers impractical
- Iterative Kaczmarz-type methods are well-suited

Existing Methods & Limitations

Tensor Kaczmarz variants in the literature:

- **TREK** – Tensor Randomized Extended Kaczmarz
- **TREBK** – Tensor Randomized Extended Block Kaczmarz
- **TREGBK** – Tensor Randomized Extended Greedy Block Kaczmarz

Key limitation:

Pre-defined Partitions Required

TREBK, TREGBK, and similar block methods require a **pre-defined partition** of the row/column indices.

- Partition quality directly affects convergence
- No principled way to choose the partition a priori
- Extra tuning overhead

TGDBEK: Key Idea

Two-step greedy selection at each iteration k:

Z-step (column selection):

- Score each column slice by its contribution
- Greedily select the set U_k of most informative columns
- Project: $\mathcal{Z}^{k+1} \leftarrow \mathcal{Z}^k - \mathcal{A}_{:,U_k,:} * (\mathcal{A}_{:,U_k,:})^\dagger * \mathcal{Z}^k$

X-step (row selection):

- Score each row slice by its residual
- Greedily select the set J_k of most informative rows
- Update: $\mathcal{X}^{k+1} \leftarrow \mathcal{X}^k + (\mathcal{A}_{J_k,:,:})^\dagger * \mathcal{R}_{J_k}$

No pre-defined partitions needed!

Algorithm 1: Tensor Greedy Double Block Extended Kaczmarz (TGDBEK)

Input: $\mathcal{A} \in \mathbb{R}^{N_1 \times N_2 \times N_3}$, $\mathcal{B} \in \mathbb{R}^{N_1 \times K \times N_3}$, iterations ℓ, parameter $\eta \in (0, 1]$.

Initialize: $\mathcal{X}^{(0)} = \mathcal{O}$ and $\mathcal{Z}^{(0)} = \mathcal{B}$.

for $k = 0, 1, \ldots, \ell - 1$ **do**

$$\varepsilon_k^z = \eta \max_{1 \le j \le N_2} \left\{ \frac{\left\| (\mathcal{A}_{:,j,:})^\top * \mathcal{Z}^{(k)} \right\|_F^2}{\left\| \mathcal{A}_{:,j,:} \right\|_F^2} \right\}$$

$$U_k = \left\{ j \in [N_2] : \left\| (\mathcal{A}_{:,j,:})^\top * \mathcal{Z}^{(k)} \right\|_F^2 \ge \varepsilon_k^z \left\| \mathcal{A}_{:,j,:} \right\|_F^2 \right\}$$

Update $\mathcal{Z}^{(k+1)} = \mathcal{Z}^{(k)} - \mathcal{A}_{:,U_k,:} * (\mathcal{A}_{:,U_k,:})^\dagger * \mathcal{Z}^{(k)}$

$$\varepsilon_k^x = \eta \max_{1 \le i \le N_1} \left\{ \frac{\left\| \mathcal{B}_{i,:,:} - \mathcal{Z}_{i,:,:}^{(k+1)} - \mathcal{A}_{i,:,:} * \mathcal{X}^{(k)} \right\|_F^2}{\left\| \mathcal{A}_{i,:,:} \right\|_F^2} \right\}$$

$$J_k = \left\{ i \in [N_1] : \left\| \mathcal{B}_{i,:,:} - \mathcal{Z}_{i,:,:}^{(k+1)} - \mathcal{A}_{i,:,:} * \mathcal{X}^{(k)} \right\|_F^2 \ge \varepsilon_k^x \left\| \mathcal{A}_{i,:,:} \right\|_F^2 \right\}$$

Update $\mathcal{X}^{(k+1)} = \mathcal{X}^{(k)} + (\mathcal{A}(J_k,:,:))^\dagger * \left(\mathcal{B}_{J_k,:,:} - \mathcal{Z}_{J_k,:,:}^{(k+1)} - \mathcal{A}(J_k,:,:) * \mathcal{X}^{(k)} \right)$

end

The TGDBEK Algorithm

Greedy Selection Criterion

Given a parameter $\eta \in (0, 1]$:

Column selection (Z-step):

$$\varepsilon_z^k = \eta \cdot \max_j \frac{\|(\mathcal{A}_{:,j,:})^\top * \mathcal{Z}^k\|_F^2}{\|\mathcal{A}_{:,j,:}\|_F^2}, \quad U_k = \left\{ j : \frac{\|(\mathcal{A}_{:,j,:})^\top * \mathcal{Z}^k\|_F^2}{\|\mathcal{A}_{:,j,:}\|_F^2} \geq \varepsilon_z^k \right\}$$

Row selection (X-step):

$$\varepsilon_x^k = \eta \cdot \max_i \frac{\|\mathcal{B}_{i,:,:} - \mathcal{Z}_{i,:,:}^{k+1} - \mathcal{A}_{i,:,:} * \mathcal{X}'^k\|_F^2}{\|\mathcal{A}_{i,:,:}\|_F^2}, \quad J_k = \left\{ i : \text{score}_i \geq \varepsilon_x^k \right\}$$

- $\eta \to 1$: fewer, most informative slices (aggressive)
- $\eta \to 0$: more slices selected (conservative)
- U_k, J_k are **determined adaptively** at each iteration

Convergence Guarantee

Theorem

Let $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ and $\mathcal{B} \in \mathbb{R}^{m \times k \times p}$. The iterates $\{\mathcal{X}^k\}$ produced by TGDBEK converge ***linearly*** *to the minimum-norm least-squares solution $\mathcal{X}^\star$:*

$$\mathbb{E}[\|\mathcal{X}^k - \mathcal{X}^\star\|_F^2] \;\leq\; \rho^k \, \|\mathcal{X}^0 - \mathcal{X}^\star\|_F^2$$

where $0 < \rho < 1$ depends on η and the spectral properties of $\mathcal{A}$.

Key points:

- Convergence to the *minimum-norm least-squares* solution
- Works for inconsistent systems
- Rate improves with well-conditioned $\mathcal{A}$ and appropriate η

Full proof available in the technical report.

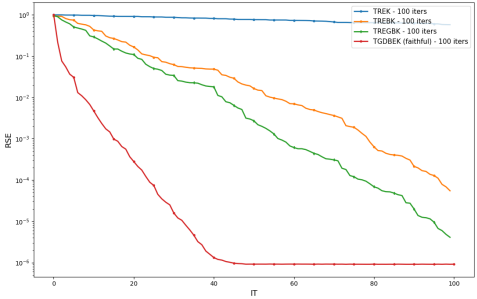
Results on Sparse Matrices (SuiteSparse)

ash85 (85×85, density $= 0.072$)

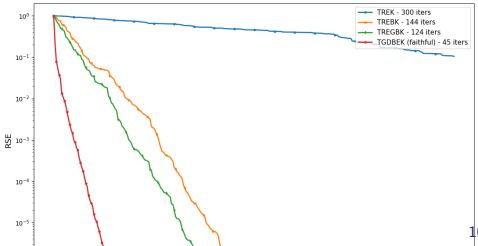
Method	**Time**	**RSE**	**IT**
TREK	0.370	2.0e−2	500
TREBK	0.369	9.2e−7	174
TREGBK	0.574	9.9e−7	258
TGDBEK	**0.248**	9.3e−7	**78**

nos5 (153×153, density $= 0.047$)

Method	**Time**	**RSE**	**IT**
TREK	0.454	1.0e−1	300
TREBK	0.860	9.8e−7	144
TREGBK	0.864	9.9e−7	124
TGDBEK	**0.581**	9.7e−7	**45**

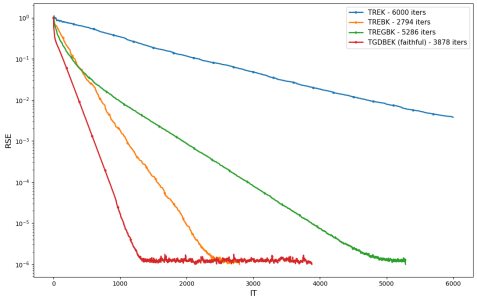


RSE vs. Iterations (ash85)



Results: Cities Matrix (128×128)

Method	Time	RSE	IT
TREK	4.089	3.8e−3	6000
TREBK	3.318	9.9e−7	2794
TREGBK	6.943	9.9e−7	5286
TGDBEK	5.954	9.9e−7	**3878**



RSE vs. Iterations (Cities)

Takeaway:

- TGDBEK consistently reaches target accuracy with **fewer iterations**
- Competitive or better wall-clock time **without needing partition tuning**

Summary & Future Work

Summary:

1. **No pre-defined partitions** – greedy selection adapts at each step
2. **Faster convergence** – fewer iterations than TREK, TREBK, TREGBK
3. **Proven guarantees** – linear convergence to min-norm LS solution

Future work:

- Larger-scale experiments (high-resolution images, video)
- GPU-accelerated implementation via PyTorch
- Extensions to other tensor decompositions (Tucker, TT)
- Applications to tensor completion and compressed sensing

Thank you!

Code:

`https://github.com/jnlandu/tensor-greedy-double-extended-kaczmarz`